

GPU Affinity: A Min-Max Work Queue for **mandelHybrid**

Matias Saavedra

Friday 10th July, 2026

Binary: `branch feat/gpu-affinity` (intended tag `binary-v5-affinity`) built on top of `binary-v3-9point` (ab47540). Baseline is `binary-v3-9point` itself — the FIFO deque.

Resumen

Report 11 showed a shared largest-first priority queue cannot route the ~ 14 s interior outlier to the GPU: with eleven CPU threads versus one GPU thread, a CPU thread extracts it $\sim 11:1$. This report fixes that with executor-aware extraction: `WorkQueue` becomes a min-max `multiset` ordered by pixel count, the GPU thread pops the largest region and CPU threads pop the smallest. Measured against the FIFO baseline (100 frames, 1920×1080 , i7-9750H + GTX 1660 Ti Max-Q, 12 threads, `diffT=0.1`, `save=0`, on AC), this is the first change to move the wall: 49.86 s vs. 51.73 s, -3.6% , statistically clean (every affinity rep beats every base rep; ranges do not overlap). The mechanism is exactly as designed: the GPU now takes the 960×540 outlier (460 ms on the GPU, CUPTI kernel max, vs. 13.3 s on a CPU thread) plus 259 interior 480×270 regions; the CPU's worst region collapses from 13.3 s to 1.34 s, its median from 88 ms to 8.7 ms, and CPU outliers drop from 1 to 0. The gain is bounded at -3.6% because the GPU is now $\sim 91\%$ utilised — near saturation — so the CPU pool (~ 49.5 s) is the new wall and further gains require work-reduction, not scheduling.

1. Setup and the Change Measured

Workload and machine.

As in reports 09 and 11: same `spec.in` (100 frames, 1920×1080 , zoom to `maxIterations=10,000`), i7-9750H (6c/12t) + GTX 1660 Ti Max-Q, 12 threads (1 GPU + 11 CPU), `diffThreshold=0.1`, `pixelSizeThresh=32,768`, `save=0`, on AC (base gate run: 51.2 s, GPU 4,235 leaves — healthy).

The change.

A FIFO deque hands every worker the same end. The new `WorkQueue` is a min-max structure — a `std::multiset` ordered by pixel count — and `extract()` takes the caller's executor type:

Código 1: `workqueue.cpp` — executor-aware extraction.

```
1 MandelRegion* WorkQueue::extract(bool isGPU) {  
2     QMutexLocker ml(&l);  
3     if (queue.empty()) return NULL;  
4     auto it = queue.begin();          // CPU threads: smallest
```

```

5  if (isGPU) { it = queue.end(); --it; } // GPU thread: largest
6  MandelRegion* r = *it; queue.erase(it); return r;
7  }

```

The GPU thread pulls the largest pending region — where it is $\sim 30\times$ faster on big coherent interior regions (report 11, §3.4) — and the CPU threads pull the smallest. `MandelRegion::Compare` was made `const` (required by `multiset`) and ascending. The decomposition is unchanged (5,608 leaves); only the dispatch order/owner changes. The graceful-degradation property: when only one size class remains, `begin()` and `end()` coincide, so neither executor idles while work exists.

Measurement.

Wall A/B (3 reps, alternating), a metrics run, a Nsight Systems capture, and a `quiet=0` distribution run, on both binaries.

2. Results

2.1. Wall Time — The First Win

Tabla 1: FIFO base vs. GPU affinity. A/B is 3 alternating reps.
The ranges do not overlap — the improvement is signal, not noise.

Metric	FIFO (base)	GPU affinity	Δ
Wall, A/B mean (s)	51.73	49.86	−1.88 (−3.6%)
A/B range (s)	51.59–51.93	49.69–49.99	disjoint
Total leaves	5,608	5,608	0
GPU / CPU	4,235 / 1,373	3,679 / 1,929	bigger→GPU
GPU kernel avg / max (ms)	2.96 / 139	5.80 / 460	GPU took the outlier
Worst CPU region (s)	13.3	1.34	outlier off CPU
CPU outliers >5 s	1	0	−1
CPU thread wall (s)	50.4–51.2	49.0–49.6	relieved

2.2. Where the Big Regions Went

Tabla 2: Region routing and the resulting per-region distributions (`quiet=0`). Affinity moves the big coherent regions to the GPU; the CPU tail collapses.

Quantity	FIFO (base)	GPU affinity
GPU took 960×540 (the outlier)	0	1
GPU took 480×270 (interior)	182	259
CPU per-region median (ms)	88.3	8.7
CPU per-region p99 (ms)	4,004	1,205
CPU per-region max (ms)	13,292	1,340
GPU per-region max (ms)	139	407

2.3. NVTX / CUPTI (Nsight Systems)

The NVTX **CPU region compute** total falls from 570.7 s to 543.7 s summed across the eleven CPU threads, and its max from 14.71 s to 1.16 s — the outlier is no longer a CPU range. **GPU region compute** is essentially unchanged (47.3 → 45.3 s busy); the CUPTI kernel count drops (4,327 → 3,424) while the average rises (2.96 → 5.80 ms) and the max jumps to 460 ms — the GPU does fewer, larger kernels, including the offloaded outlier.

3. Analysis

3.1. The win is real, and it is the heterogeneity that delivers it

The A/B ranges are disjoint (Table 1: base 51.59–51.93 s vs. affinity 49.69–49.99 s), so the -3.6% is signal. This is the first optimisation in the series to move the wall, and the reason is precisely what reports 09 and 11 said was missing: those changes only rebalanced equal-cost work on a throughput-bound, already-balanced schedule, which cannot help. GPU affinity instead performs heterogeneous assignment — it moves the expensive coherent interior regions onto the processor that is $\sim 30\times$ faster on them. The 960×540 outlier costs 13.3 s on a CPU thread but 460 ms on the GPU (Table 2); routing it (and 77 more interior 480×270 regions) to the GPU relieves the CPU pool, whose per-thread wall drops from 50.4–51.2 s to 49.0–49.6 s and whose worst region collapses from 13.3 s to 1.34 s.

3.2. Why it works where the shared priority queue (report 11) did not

Report 11’s largest-first **priority_queue** exposed only one end, so all twelve workers competed for the biggest region and a CPU thread won it $\sim 11:1$ — the outlier stayed on CPU (kernel max unchanged at 139 ms). The min-max queue makes extraction executor-aware: the GPU thread alone pulls the largest, the CPU threads pull the smallest, so the big regions are effectively reserved for the GPU. The kernel max moving from 139 ms to 460 ms (Table 1) is the direct fingerprint of the outlier finally landing on the GPU.

3.3. Why the win is bounded at -3.6% : the GPU is now near-saturated

The GPU’s busy time is ~ 45 s of the 49.86 s wall — about 91 %. It cannot absorb much more, and the CPU pool (~ 49.5 s) is now the binding wall. The two processors are balanced near 49.9 s. Pushing lower can no longer come from moving work — both lanes are busy — only from reducing it. Total compute is still $\sim$ conserved; the lever from report 09 (periodicity checking in **diverge()**, or an interior certificate that skips iterating in-set pixels) remains the way past this floor. GPU affinity has taken the schedule about as far as scheduling alone can.

3.4. The CPU-only guard: implemented and measured

With the GPU disabled, `thread 0` is also a CPU worker, so without a guard all threads would pull the smallest — smallest-first, which defers the big regions to the tail. A guard was added: when `gpuEnable=0` (`gpuPresent` false), CPU threads pull the largest instead (longest-processing-time, LPT). To check both the feared regression and the guard’s value, the three CPU-only orderings were run head to head (`gpu=0`, 12 threads, 2 reps each).

Tabla 3: CPU-only (`gpu = 0`, 12 threads) under three queue orderings. All three are within $\sim 0.4\%$ — at this scale the ordering is a wash.

Ordering	Wall mean (s)	Worst region (s)	Leaves
FIFO (baseline)	156.84	14.1	5,608
Smallest-first (no guard)	157.41	14.0	5,608
Largest-first / LPT (guard)	156.88	14.3	5,608

The result is a wash: all three orderings land at ≈ 157 s, within $\sim 0.4\%$ (per-thread spread under 1% in every case). The feared smallest-first regression does not materialise ($+0.4\%$, noise), and LPT does not beat FIFO either. The reason is the same throughput-bound property as the hybrid: with 5,608 fine-grained leaves spread over 12 threads (~ 467 each), the tail is always well-filled, so when a region runs barely matters — the 14 s outlier is only 9% of the wall and overlaps hundreds of other regions regardless of order. Scheduling order only moved the wall when it changed which processor ran the work (the hybrid GPU affinity); on a homogeneous CPU pool it cannot. The guard is therefore kept as the principled, zero-cost choice — it protects smaller-scale or coarser workloads where the tail is not self-filling — but it earns nothing measurable on this one.

4. Recommendations

1. Adopt GPU affinity as the default work queue. It is a clean -3.6% (-1.88 s) with disjoint A/B ranges, eliminates the CPU outlier (worst region $13.3 \rightarrow 1.34$ s), is decomposition-identical, and is contained to `WorkQueue`. Merge `feat/gpu-affinity` to `main`.
2. Keep the CPU-only guard (§4.4). It is implemented (CPU threads pull largest when no GPU) and is the principled choice, though measured to be a wash at this workload scale — it costs nothing and serves both configurations safely.
3. Next, attack work, not schedule (report 09). With the GPU near-saturated and the CPU pool the wall, the only remaining lever is reducing the ~ 595 s of compute: periodicity checking in `diverge()` (exact) or an interior certificate. Scheduling has been exhausted.

5. Conclusion

Making work-queue extraction executor-aware — GPU thread pulls the largest region, CPU threads the smallest — routes the expensive coherent interior regions, including the ~ 14 s outlier, onto the $\sim 30\times$ -faster GPU (460 ms there). This is the first change in the series to move the end-to-end wall: -3.6% ($51.73 \rightarrow 49.86$ s) with disjoint A/B ranges, and the CPU outlier is eliminated (worst CPU region $13.3 \rightarrow 1.34$ s, zero >5 s regions). It succeeds where report 11’s shared priority queue failed because it assigns by executor, not by a single shared order that the eleven CPU threads win 11:1. The win is bounded at -3.6% because the GPU is now $\sim 91\%$ utilised: scheduling has been taken to its limit, and the remaining lever is work-reduction.